TorrentZip

This specification is intended to define the implementation of zip as used by the TorrentZip standard.

Archive format

General format of a torrentzipped .zip file with n files:

Archive Start
[local file header 1]
[file data 1]
[local file header 2]
[file data 2]
…
[local file header n]
[file data n]
Central Directory - Start (SOCD file offset)
[central directory file 1]
[central directory file 2]
…
[central directory file n]
Central Directory - End (EOCD file offset)
[end of central directory record]
Archive End

Local file header x: (Showing torrentzipped default values):

Type	Attribute	Value	Description
UInt32	Local file header signature	(0x04034b50)	
UInt16	Version needed to extract	20 or 45	20 = File is compressed using Deflate compression / 45 if this record contains zip64 information
UInt16	General purpose bit flag	2	Maximum compression option was used, bit 11 (0x800) is set for unicode filename
UInt16	Compression method	8	The file is Deflated
UInt16	Last mod file time	48128	11:32 PM
UInt16	Last mod file date	8600	12/24/1996
UInt32	CRC32		File CRC32

Type	Attribute	Value	Description
UInt32	Compressed size		File Compressed Size
UInt32	Uncompressed size		File Uncompressed Size
UInt16	Filename length		Filename length
UInt16	Extra field length		Normally 0, Length of Extra field data if zip64 extra field information is included
Byte[]	Filename (variable size)		Byte array of filename

> The default values are required to have consistent torrentzipped files. Default time/date of 11:32pm 12/24/1996 is the date of the first ever MAME release.

File data x:

The data compression must be exactly as ZLib version 1.1.3 using maximum compression level 9. All files and empty directories must be compressed with the Deflate method, which is method 8.

Central Directory file x: (Showing torrentzipped default values):

Type	Attribute	Value	Description
UInt32	Central file header signature	(0x02014b50)	
UInt16	Version made by	0	MS_DOS and OS/2 (FAT/FAT32 file systems)
UInt16	Version needed to extract	20 or 45	20 = File is compressed using Deflate compression / 45 if this record contains zip64 information
UInt16	General purpose bit flag	2	Maximum compression option was used, bit 11 (0x800) is set for unicode filename
UInt16	Compression method	8	The file is Deflated
UInt16	Last mod file time	48128	11:32 PM
UInt16	Last mod file date	8600	12/24/1996
UInt32	CRC32		File CRC32
UInt32	Compressed size		File Compressed Size
UInt32	Uncompressed size		File Uncompressed Size
UInt16	File name length		Filename length
UInt16	Extra field length		Normally 0, Length of Extra field data if zip64 extra field information is included
UInt16	File comment length	0	No file comment
UInt16	Disk number start	0	Multi disk storage not used so set to disk 0
UInt16	Internal file attributes	0	No internal attributes
UInt32	External file attributes	0	No external attributes
UInt32	Relative offset of local header		File offset of this files Local Header

Type	Attribute	Value	Description
Byte[]	File name (variable size)		Byte array of filename

End of Central Directory:

Type	Attribute	Value	Description
UInt32	End of central dir signature	(0x06054b50)	
UInt16	Number of this disk	0	Multi disk storage not used so set to disk 0
UInt16	Number of the disk with the start of the central directory	0	Multi disk storage not used so set to disk 0
UInt16	Total number of entries in the central directory on this disk	n	Total number of files
UInt16	Total number of entries in the central directory	n	Total number of files
UInt32	Size of the central directory	EOCD-SOCD	length of the central directories
UInt32	Offset of start of central directory with respect to the starting disk number	SOCD	Start of central directory
UInt16	.ZIP file comment length	22	torrentzipped comment
Byte[22]	.ZIP file comment	TORRENTZIPPED-XXXXXXXX	

> See above 'General format of a torrentzipped .zip file with n files' for SOCD & EOCD

The TorrentZipped files comment

The .ZIP file comment in the End of Central Directory is used to check the validity of the torrentzipped file.

The comment must be formatted as the 22 bytes of `TORRENTZIPPED-XXXXXXXX`. The `XXXXXXXX` is the CRC32 of the central directory records stored as hexadecimal upper case text. (The CRC32 of the bytes in the file between SOCD & EOCD)

This comment ensures that if any change is made to the files within the zip this checksum will no longer match the byte data in the central directory, and in this way we can check the validity of a torrentzip file.

Filename Encoding

The filenames of the compressed files in a zip file are stored in the local header and the central directory as byte arrays. Zip was original build on early IBM PCs, and as such uses code page 437 [https://en.wikipedia.org/wiki/Code_page_437] to convert a string to a byte array to store the filenames. With the arrival of unicode multiple different methods where added to the official zip format to permit unicode filenames to be stored in a zip file. Trrntzip format uses the general purpose bit 11 method. So to store a filename in a trrntzip zip file you must first see if the filename can be stored using code page 437, if not then UTF8 encoding should be used in the byte arrays, this is then indicated by setting bit 11 of the General Purpose Bit Flags both in the local header and central directory.

File order with a TorrentZip

For the creation of consistent torrentzipped files, the file order is also very important. Files must be sorted by filename using a lower case sort.

Directory separator character

As zips only store files (not directories), files in directories are represented by storing a relative path to the filename. For example file `test1.rom` in directory `set1` would be stored with a filename of `set1/test1.rom`. Some zipping programs will store this as `set1\test1.rom`.

This leads to a possible naming inconsistency. The zip file format states: *"All slashes should be forward slashes '/' as opposed to backwards slashes '\'"*. So Torrentzip will change all `\` characters to `/`. This must be done before sorting, to ensure that the sort is performed correctly.

Directory entries and empty directories

A directory entry is stored in a zip by adding a file entry ending in a directory separator character with a zero size and CRC. So directory `set1` would be stored as a zero length, zero CRC file called `set1/`. Some zip programs when adding the previously mentioned file `set1/test1.rom` will also add the directory `set1/`, this creates an inconsistency problem. In this example the `set1/` directory entry is unnecessary, as the filename `set1/test1.rom` implies the existents of the `set1/` directory. To resolve this inconsistency un-needed directories should be removed from the zip, the only needed directory entries are empty directories that are not implied by any file entries.

Example:

Filename	Size	CRC
set1/	0	00000000
set1/test1.rom	1024	53AC4D00
set2/	0	00000000

The `set1/` entry should be removed, as it is implied by the `set1/test1.rom` file. The `set2/` entry should be kept to create the empty directory, as removing it would completely remove the `set2` directory.

Repeat files

Another test that could be performed is checking for repeat file entries inside the zip, most zip programs have a hard time handling this and will just ignore this repeat giving the user no way of knowing there is a repeat filename problem. So it would fix another possible inconsistency if torrentzip scanning at least warned about repeat filename being found inside a zip.

Reference

- Official ZIP file specification [http://www.pkware.com/documents/casestudies/APPNOTE.TXT]
- Original TorrentZip source code [http://sourceforge.net/projects/trrntzip/]
- zlib compression [http://zlib.net/]